

CompanyDocuments: Document Generator Design

Design notes for the synthetic document generator behind the `AyoubChLin/CompanyDocuments` dataset, built on Northwind data.

1. Core idea: three separate layers

Most generators end up as one template per document type, which caps diversity quickly. Splitting the work into layers multiplies it instead:

Layer	Responsibility
Data	Northwind rows become a canonical, validated record (parties, addresses, line items, totals, dates). Templates never touch the database.
Layout	A layout family decides where components go: header, party blocks, line table, totals, footer, notes.
Style	A theme sets fonts, colors, table style, logo, and spacing.

Each document is a combination of `doc_type` × `layout` × `theme` × `locale` × `content variation`, so 4 layouts × 5 themes × 3 locales already gives 60 looks per document type.

2. Decisions that matter

Rendering engine

HTML + Jinja2 → WeasyPrint. It is pure Python and runs fine on a Linux laptop with no GPU. It handles CSS tables and automatic page breaks (multi-page invoices come for free) and shapes Arabic and other complex scripts properly. ReportLab is more precise but every layout becomes code. Chromium/Playwright has the best CSS but is heavy for thousands of documents.

Components as macros

Build a small component library (`header`, `party_block`, `line_table`, `totals`, `footer`, `stamp`) and let layouts compose them differently. A new document type is then mostly a new composition, not a new template.

Vary the issuer

In Northwind every document comes from the same seller, so every header would be identical and models would learn to recognize it. Generate multiple issuer companies (name, address, logo, tax ID, bank details) and treat Northwind customers and suppliers as counterparties.

Content variation

- Optional fields appear or vanish: discount column, PO reference, notes, second address line.
- Line items range from 1 to 40, so some documents run to several pages.

- Tax can be single-rate, multi-rate, or absent.

Numbers must be correct

Use `Decimal` and compute subtotal, discount, freight, tax, and total in the data layer with explicit rounding rules. If currencies are converted (USD to EUR, for example), convert once and compute from there. The gold JSON includes these computed values, which also enables arithmetic-consistency validation later.

Gold labels and boxes for free

Gold labels come from the canonical record. For word boxes, extract words from the rendered PDF with PyMuPDF and align them to the gold values, so you get layout supervision without OCR errors.

Reproducibility

Use one seed per document and `doc_id = hash(order_id, type, layout, theme, locale, seed)`. The whole run is described by a YAML config, so v2.1 can regenerate or extend v2.0 exactly.

Separate the noise stage

Render clean born-digital PDFs first. Scan and photo augmentation is a second stage on the rasterized pages, so both the clean and the degraded version of the same document are kept.

Self-check

After rendering, re-extract the text and verify that every gold field appears and that the totals add up. Documents that fail are dropped, so bad rows never reach the dataset.

3. Suggested first milestone

Prove the pipeline end to end before scaling it:

- **3 document types:** invoice, purchase order, packing slip
- **Variety:** 4 layouts × 3 themes
- **2 locales:** English and French
- **Volume:** about 500 documents, with the self-check passing

After that, adding a type or a language is incremental work, and the heavier types (commercial invoice, account statement, credit note) come in.

4. Open question

Which languages should come first? English and French are the default suggestion; Arabic would add right-to-left layout on top of that.